

Vision Models: From Pixels to Diffusion

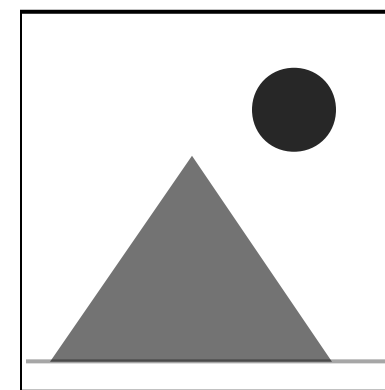
An image is just a tensor of numbers

A color image is a grid of pixels: **height** × **width** × **3 channels** (R, G, B), each value 0—255

A small 224×224 photo is already ~150,000 numbers — meaning lives in **patterns**, not pixels

The same object shifts, scales, rotates, changes lighting — the model must learn **invariance**

Goal: compress raw pixels into a short, meaningful vector — an **embedding**

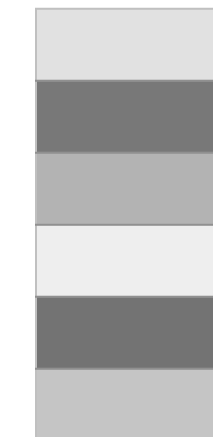


WHAT WE SEE



68	159	147	41	102	242
162	129	168	156	24	163
11	240	222	128	74	149
67	57	191	128	146	222
148	129	109	171	228	46
67	170	46	230	245	141

WHAT THE COMPUTER SEES
H × W × 3 · VALUES 0-255



EMBEDDING
512 numbers

Convolution: a small filter slides over the image THEORY

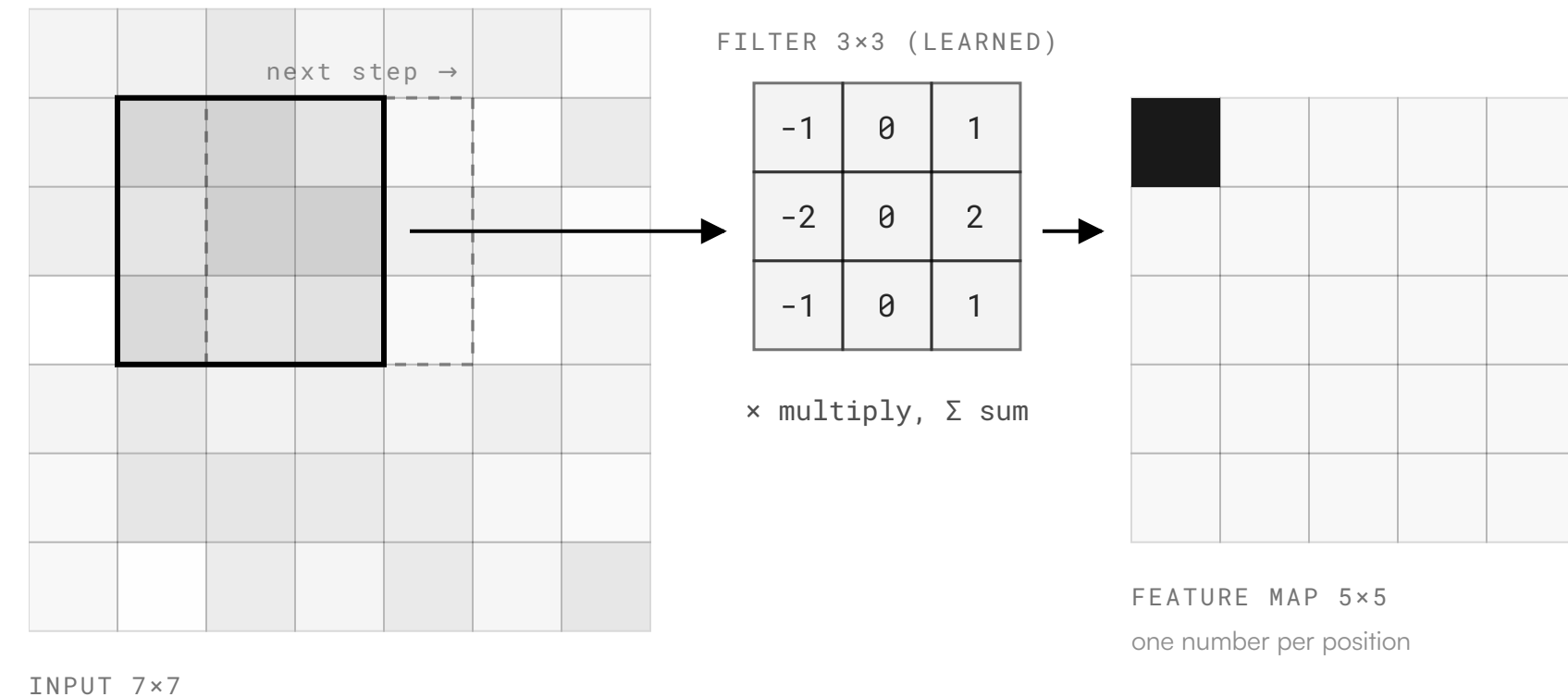
A **filter (kernel)** is a tiny grid of learned weights — e.g. 3×3 — slid across the image

At each position: multiply weights by pixels underneath, sum $\rightarrow$ **one output number**

The output grid is a **feature map** — it lights up wherever the filter's pattern appears

Weight sharing: same filter everywhere $\rightarrow$ few parameters; pattern found at any location

Stack layers: edges $\rightarrow$ textures $\rightarrow$ object parts $\rightarrow$ objects



Pooling + stacking = a CNN classifier

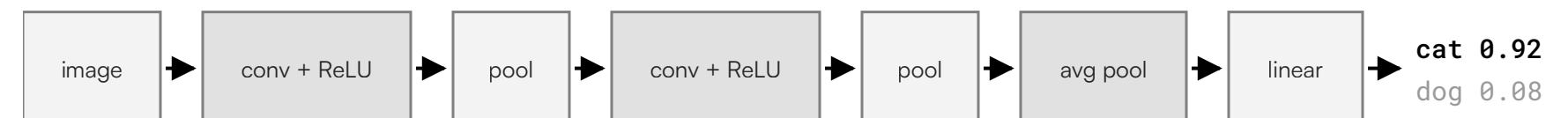
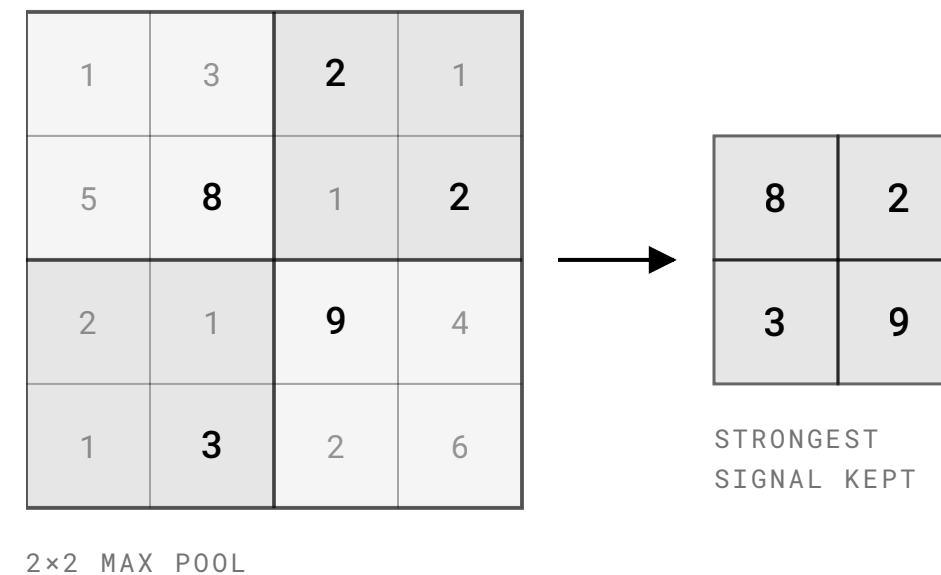
Max pooling: keep the largest value in each 2×2 window — halve resolution, keep the strongest signal

Average pooling: take the mean — used globally at the very end of most CNNs

Pooling adds tolerance to small shifts and makes deeper layers cheaper

Recipe: $[\text{conv} \rightarrow \text{ReLU} \rightarrow \text{pool}] \times N$, then a linear layer $\rightarrow$ class probabilities

In code: `nn.Conv2d`, `nn.MaxPool2d`, `nn.Linear` — that's a full classifier



Augmentation: free data from the data you have

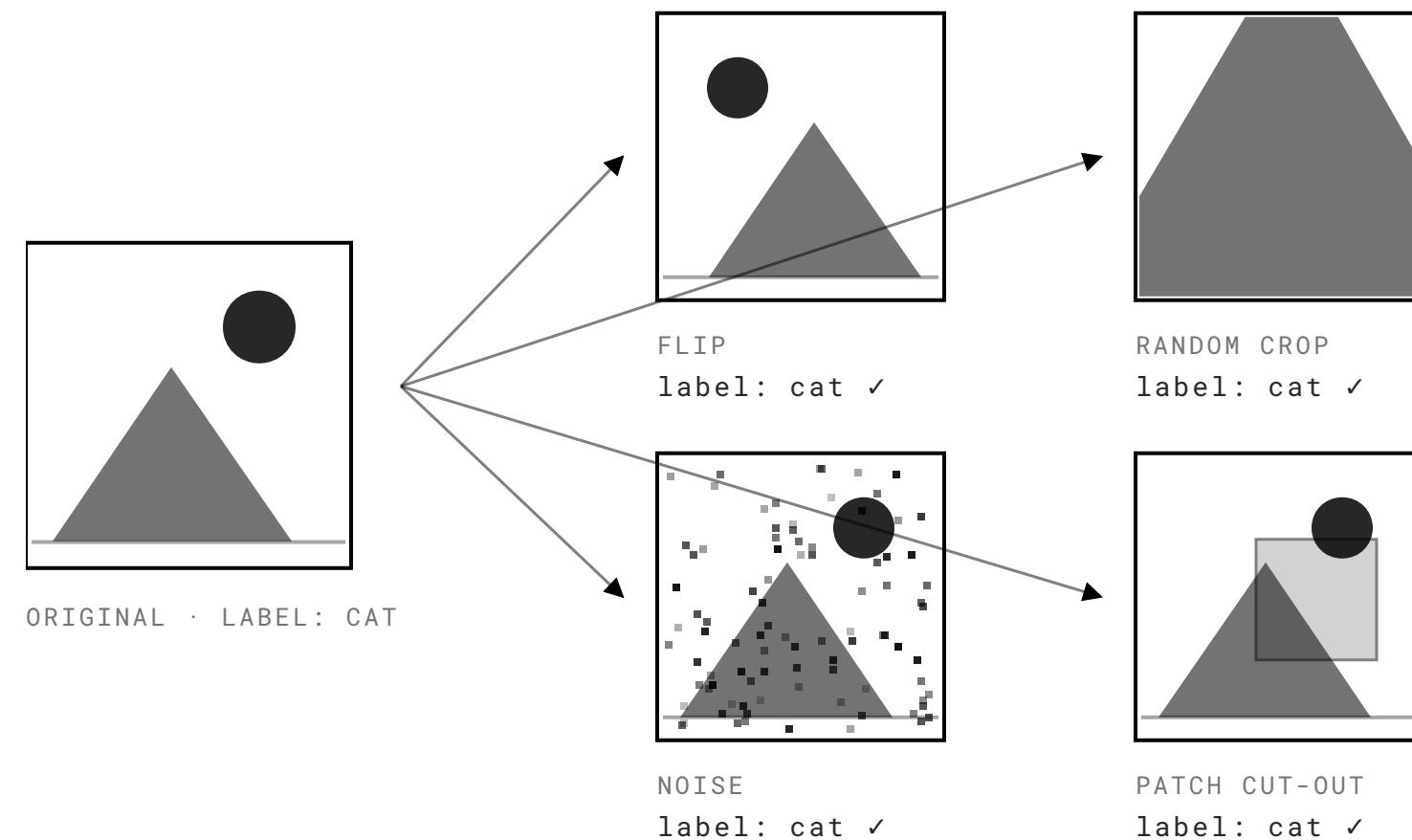
Apply label-preserving changes during training: **flip**, **random crop**, **noise**, rotation, color jitter

Model sees a “new” image every epoch → less overfitting, better generalization

Patch cut-out: hide random patches so the model can't rely on one local cue

One line per transform (torchvision.transforms) — often the best accuracy boost in competitions

Rule: the label must stay true — don't flip a “6” into a “9”



ResNet: skip connections made depth work

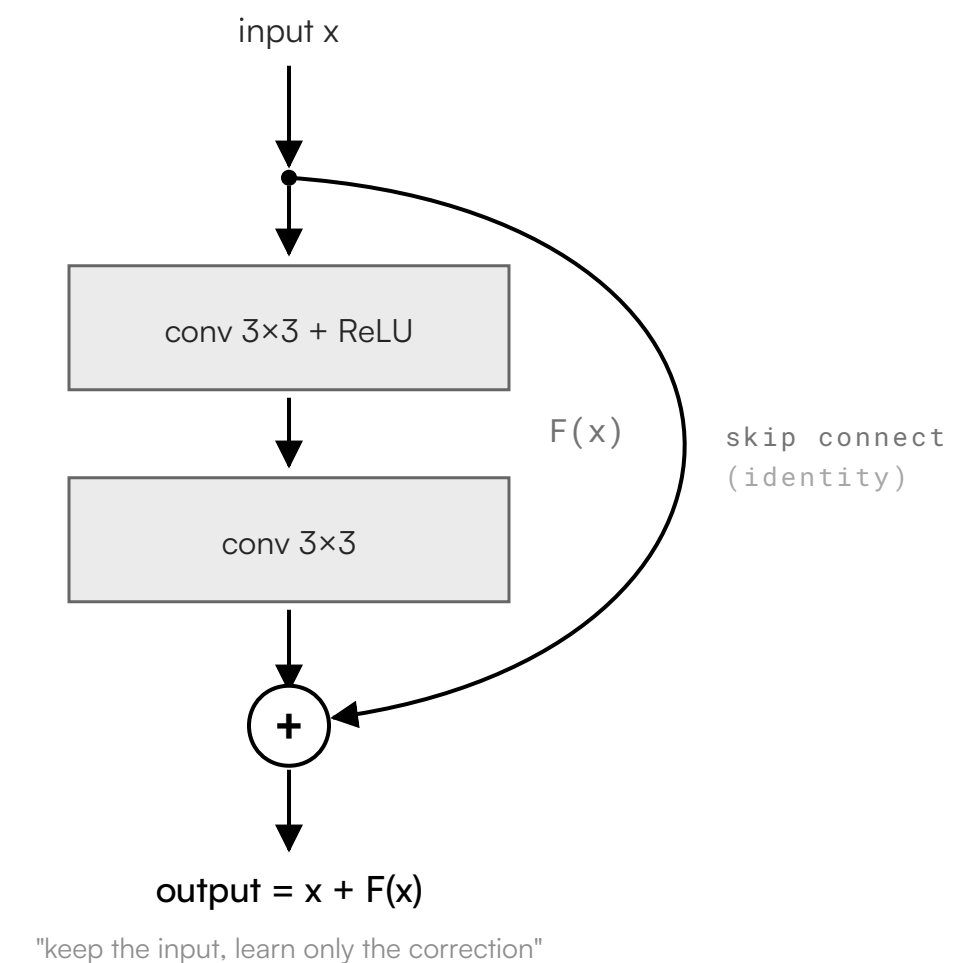
Plain deep CNNs became **hard to train** — the training signal degrades through many layers

ResNet's fix: each block outputs $x + F(x)$ — the input plus a learned correction

Gradients flow straight through the shortcut → 50—152-layer networks train reliably

Trained on ImageNet's millions of photos, ResNet became the standard **pre-trained encoder**

One line: `torchvision.models.resnet50(weights=...)` — reusable features for any task



Finetuning: start smart, not from scratch

Transfer learning: take a pre-trained encoder, swap the last layer for your classes

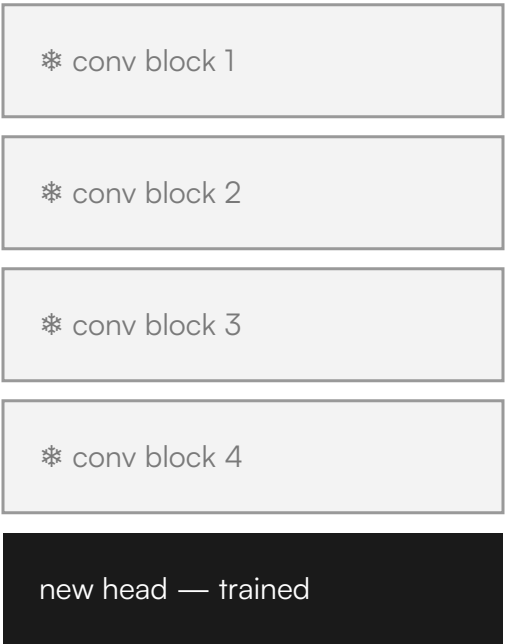
Tiny data → **freeze** the backbone, train only the new head

More data → **finetune everything** gently, with a low learning rate

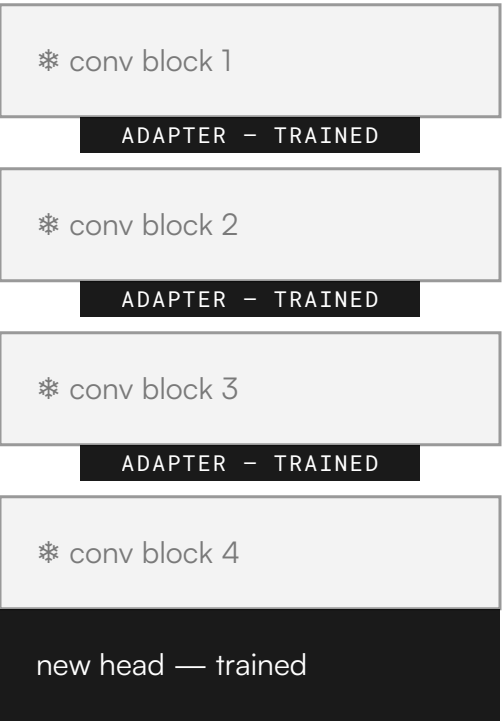
Parameter-efficient finetuning: train small inserted adapters (LoRA-style); backbone stays frozen

Why it wins competitions: small datasets + pre-trained features beat training from zero

A · FREEZE + NEW HEAD



B · PARAMETER-EFFICIENT (ADAPTERS)



~1-5% OF WEIGHTS UPDATED

Detection: what is where — YOLO, SSD, DETR

Detection = classification + **localization**: a box, label, and confidence per object

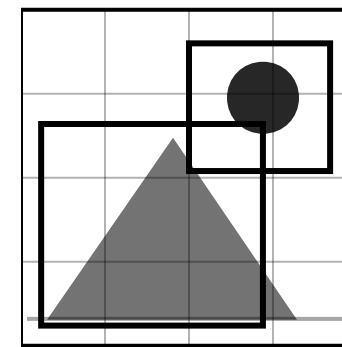
YOLO: one single pass predicts boxes over a grid — built for real-time speed

SSD: single pass, predicts from feature maps at several scales
→ small and large objects

DETR: a transformer outputs a direct **set** of detections — no anchor boxes, no NMS cleanup

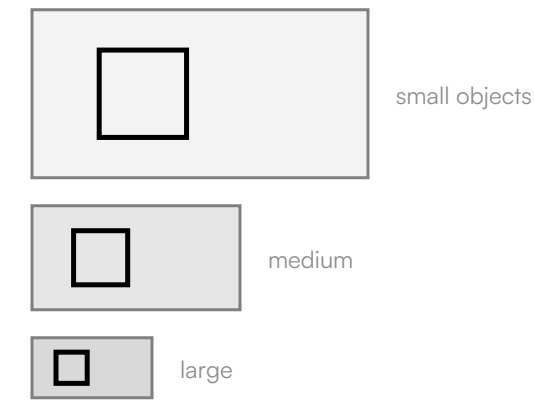
Pick: YOLO for speed • SSD classic multi-scale • DETR clean end-to-end

YOLO · GRID, ONE PASS



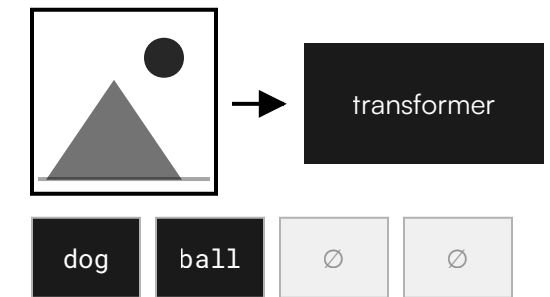
boxes from grid cells
built for real time

SSD · MULTI-SCALE, ONE PASS



boxes at several scales

DETR · TRANSFORMER SET



direct set of predictions,
no anchors, no NMS

Segmentation: a label for every pixel — U-Net

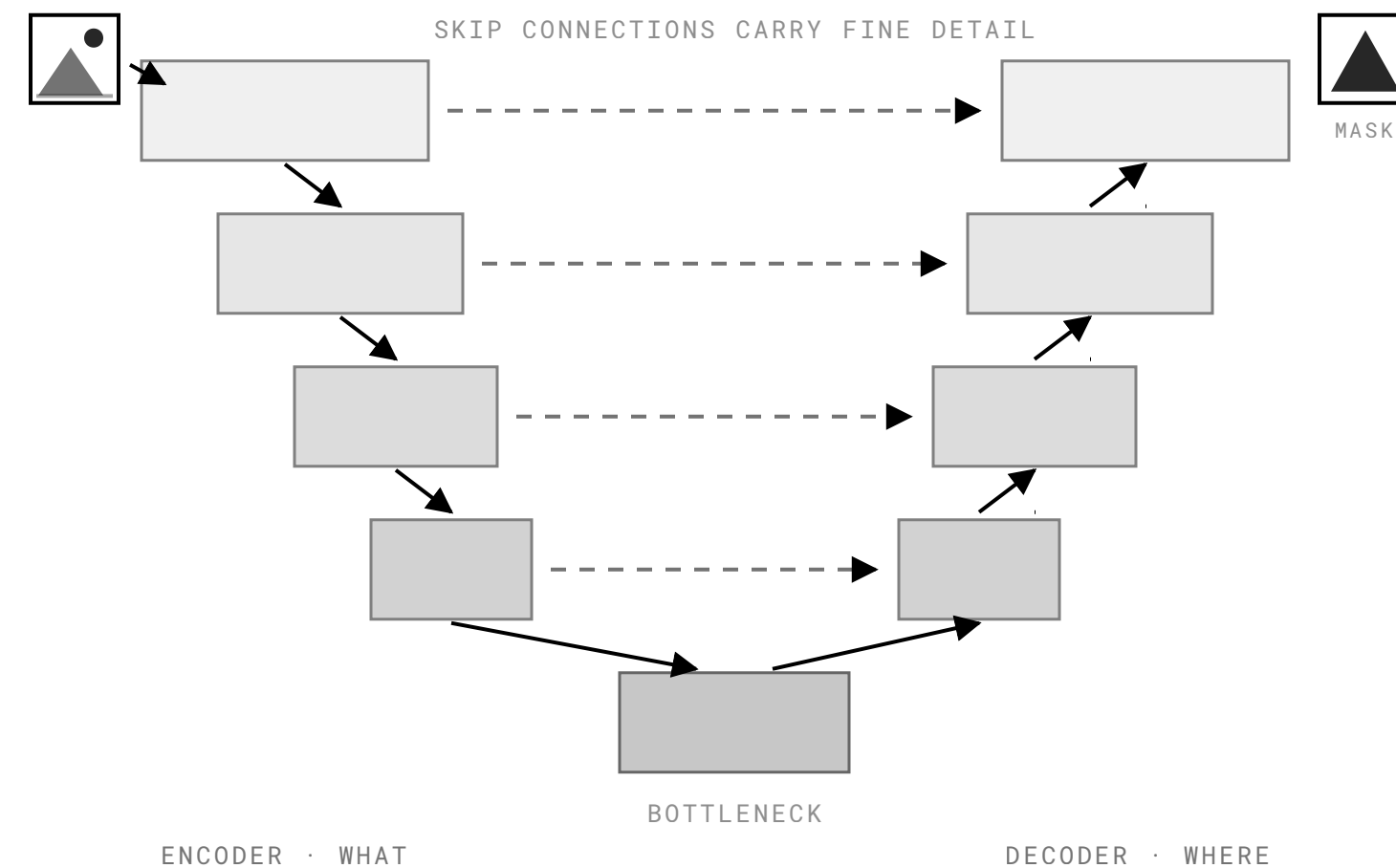
Output is a full-resolution **mask**: each pixel gets a class

Encoder downsamples to understand **what**; **decoder** upsamples to recover **where**

Skip connections copy fine detail across the U → sharp mask boundaries

Born in medical imaging; works well even with few training images

Same skip idea as ResNet — here the skips carry spatial detail



ViT: cut the image into patches, treat them like words

THEORY

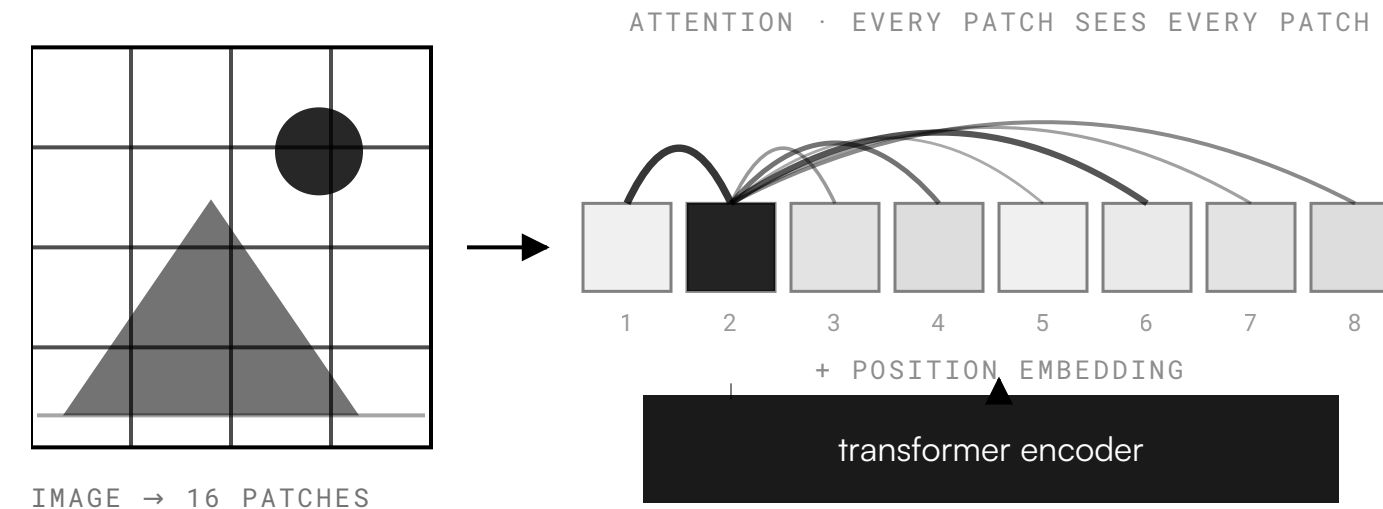
Attention: each element scores its relevance to every other element, then gathers info by those weights

Mechanic: each patch emits a **query**, compares to all **keys**, mixes the **values**

ViT: slice the image into 16×16 patches $\rightarrow$ embed each as a token $\rightarrow$ add position $\rightarrow$ transformer

Global from layer one: patch 1 can directly attend to patch 196 — unlike a conv's local window

Trade-off: no built-in locality bias $\rightarrow$ ViTs need large-scale pre-training to shine



Learning features without labels

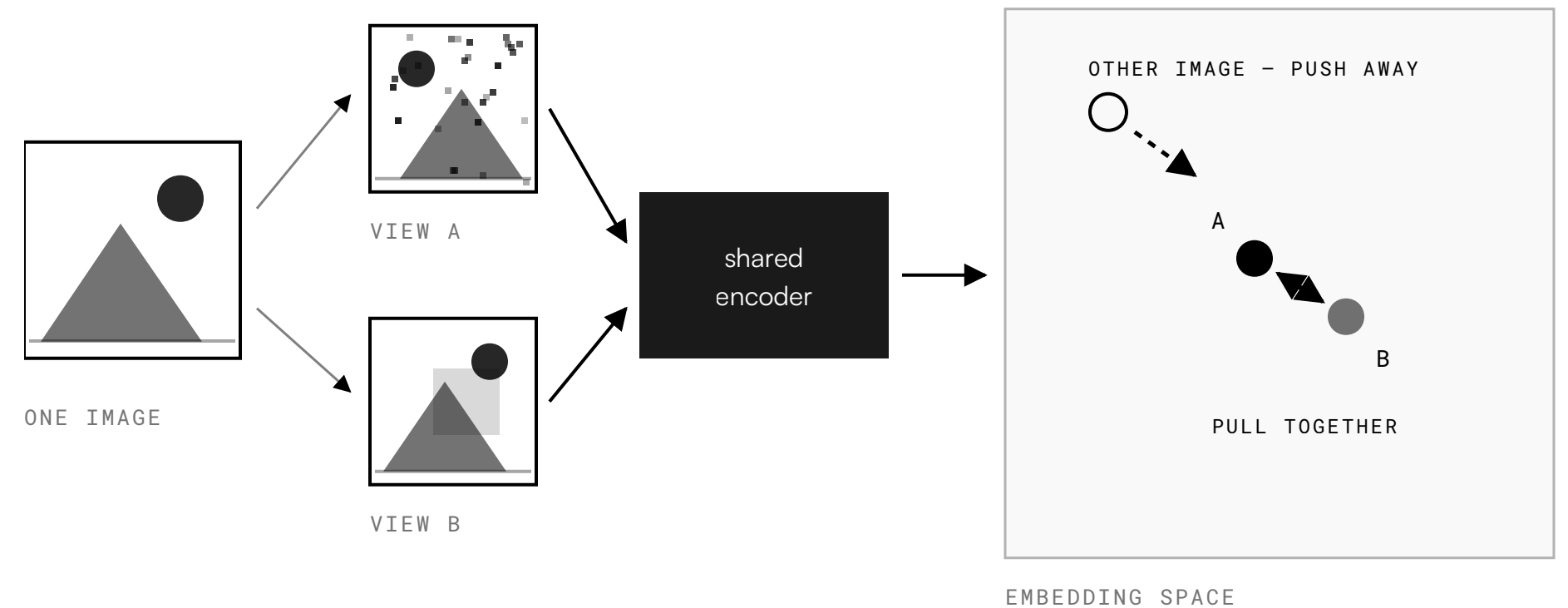
Human labels are expensive and slow; unlabeled images are nearly infinite

Self-supervision: invent a training task from the data itself

Contrastive idea: two augmented views of one image → pull embeddings together; other images → push apart

Augmentations (slide 5) define what the model should **ignore**

Result: a strong encoder from raw images — then finetune on your small labeled set



CLIP: images and text in one shared space

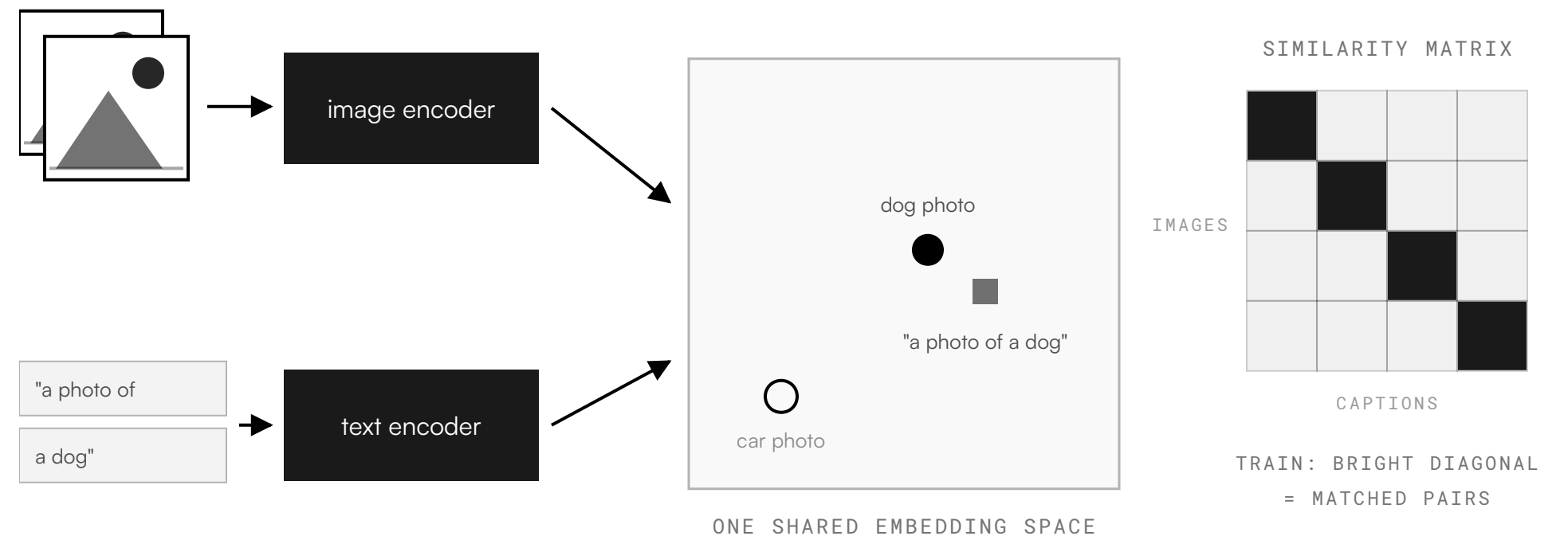
Two encoders — image and text — trained on huge sets of (image, caption) pairs

Contrastive: matching pairs → similar embeddings;
mismatched → dissimilar

Result: **“a photo of a dog”** (text) lands next to actual dog photos

Zero-shot classification: embed candidate captions + the image, pick the closest — no training

Also powers image search and guides text-to-image generators



GANs: a forger and a detective in competition

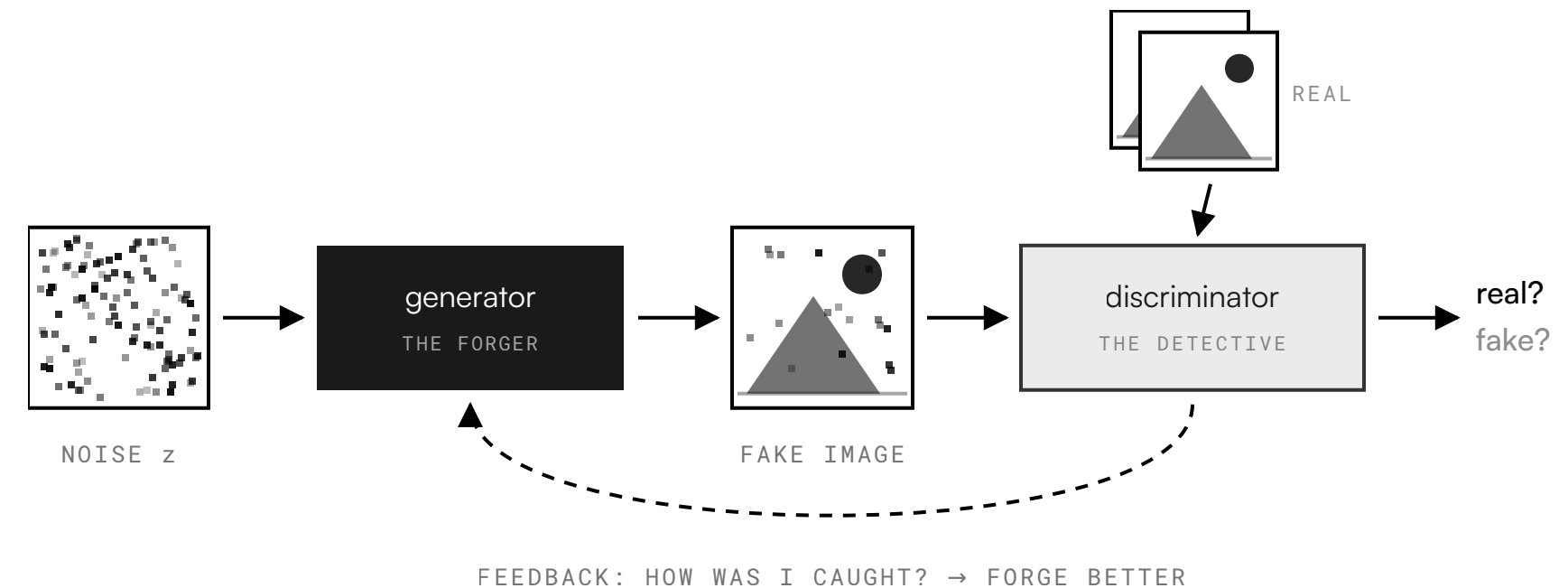
Generator turns random noise into images; **discriminator** judges real vs. fake

Adversarial loop: better detection forces better forgery — both improve together

At equilibrium, fakes become hard to distinguish from real photos

Strengths: sharp images, **one-pass** fast generation

Pain points: unstable training, **mode collapse** (only a few “safe” outputs)



Diffusion: learn to denoise, then start from pure noise

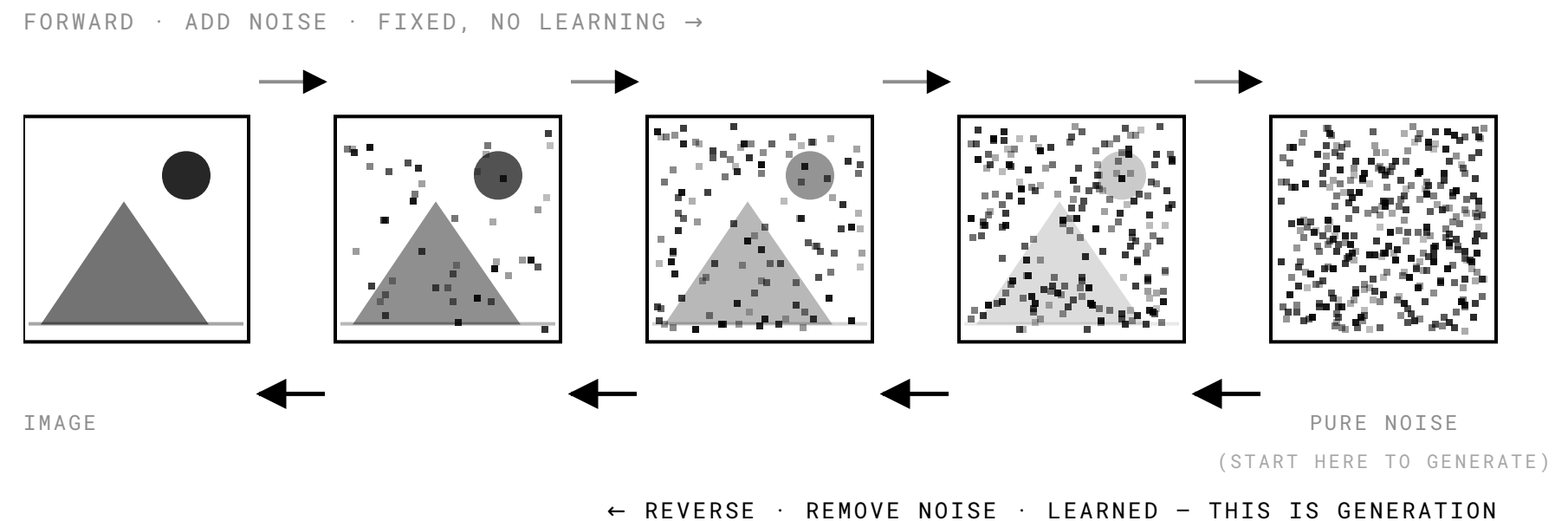
Forward (fixed): add a little noise to a real image, step by step, until pure static

Reverse (learned): a network predicts the noise so each step can subtract it

Generation: start from random static, denoise step by step → a new image condenses out

Condition on text embeddings (CLIP-style) → text-to-image, e.g. Stable Diffusion

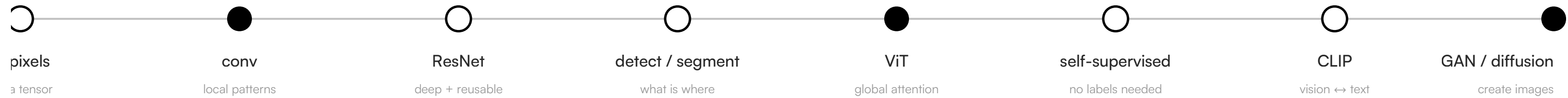
vs. GANs: stabler training, more diverse outputs — but many steps, so slower



Where each topic sits in the syllabus

TOPIC	LEVEL REQUIRED	SLIDE
Convolutional layers	THEORY + PRACTICE	03
Image classification (+ pooling)	PRACTICE	04
Image augmentation — flip / crop / noise / patching	PRACTICE	05
Pre-trained vision encoders (ResNet)	PRACTICE	06
Model finetuning	PRACTICE	07
Object detection — YOLO, SSD, DETR	PRACTICE	08
Image segmentation — U-Net	PRACTICE	09
Attention / Transformers → ViT, image embeddings	THEORY (SUPPORTING)	10
Self-supervised learning for vision	PRACTICE	11
Vision—text encoders (CLIP)	PRACTICE	12
Generating images with GANs	PRACTICE	13
Diffusion models	PRACTICE	14

From pixels to diffusion — and what I'd build next



TWO THEORY ANCHORS

Convolution and attention — be able to explain both on paper, cold.

ONE WINNING STRATEGY

Pre-trained encoder + strong augmentation + careful finetuning.

NEXT PROJECT

Finetune ResNet on a small custom dataset; compare against CLIP zero-shot as the baseline.